

Day 3

Test the Artifact

You've Built

6 hours · 3 modules · 3 labs · same repo as Days 1-2

Where we left off

Days 1-2: a working catalog, a typed API, a Python client, a bulk-import flow.

Today: **lock it all under a real test suite**. By 6 PM, CI is green.

<u>Day 2</u>	<u>Day 3 (today)</u>
server	→ pytest suite over models + catalog
client	→ parametrized + mocked HTTP tests
	+ integration class hitting a live server
	+ HTML report + GitHub Actions

Catch-up: `cp -r ../checkpoints/day-3-start/. .`

Today's arc

Module	~40 min	80 min lab
1	pytest basics + fixtures	Lab 7: Unit tests for catalog core
2	Parametrize + mocking	Lab 8: Test the APIClient
3	Reports + CI/CD	Lab 9: HTML report + GitHub Actions

End-of-day: `pytest -q` green locally, CI green on `git push`.

Module 1

Testing Fundamentals + pytest Basics

~40 min · then 80 min lab

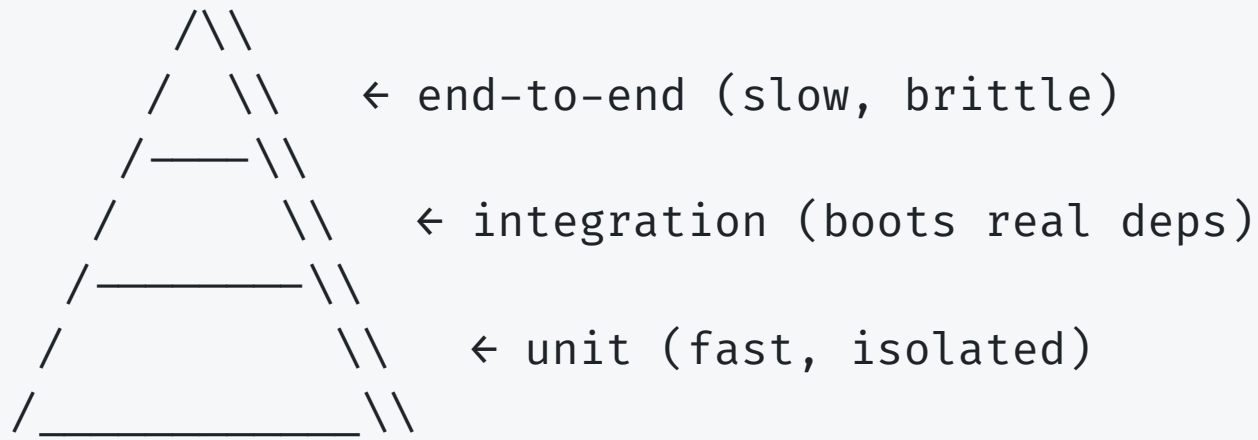
Why we test at all

Three things tests buy you, in priority order:

1. **A safety net for change.** You can refactor with confidence.
2. **Living documentation.** A reader sees the contract, not just the code.
3. **Fast feedback.** Find a bug in 0.5s, not 30 min into a deploy.

Tests are not extra work. They're the work — written down so it stays done.

The test pyramid



- **Unit** (this lab) — one function/class at a time, no network, no DB.
- **Integration** (Lab 8) — your code + a real collaborator (FastAPI process).
- **E2E** (out of scope) — through the browser / full stack.

Rule of thumb: 10 unit : 3 integration : 1 e2e.

pytest in 30 seconds

```
def test_addition():  
    assert 2 + 2 == 4
```

That's it. Save as `test_*.py`, run `pytest`. No `setUp/tearDown` ceremony, no `unittest.TestCase` base class. Just functions.

Grouping is optional but useful:

```
class TestProductValidation:          # plain class, no inheritance  
    def test_valid_payload(self): ...  
    def test_rejects_empty_name(self): ...
```

Fixtures — share setup, don't repeat it

```
@pytest.fixture
def seeded_catalog() → ProductCatalog:
    return ProductCatalog([
        Product(id=10, name="Cable", category="Electronics", price=499.0),
        Product(id=11, name="Speaker", category="Electronics", price=2499.0),
        Product(id=12, name="Mat", category="Fitness", price=1299.0),
    ])
```

Any test that names `seeded_catalog` as a parameter gets a **fresh** one.

```
def test_search(seeded_catalog):
    assert seeded_catalog.search_by_name("cable")[0].id == 10
```

Fixture scope: `function` (default) → `class` → `module` → `session`. Default to the narrowest.

`conftest.py` — shared across files

```
tests/  
├── conftest.py           ← fixtures visible to ALL test files below  
├── test_models.py  
├── test_catalog.py  
└── test_client.py
```

No imports needed in test files — pytest auto-discovers fixtures defined in `conftest.py`.
Magic? A little. Convention? Strong.

Asserting on errors

```
import pytest

with pytest.raises(CatalogError, match="already exists"):
    catalog.add(duplicate)
```

Three things at once:

1. The block **must** raise an exception
2. The exception **must** be `CatalogError` (or a subclass)
3. Its message **must** contain "already exists"

`match=` is regex. Always pass it — otherwise a wrong-but-same-type error passes the test.

Test data organisation

```
# Good – one fixture per concept
@pytest.fixture
def sample_product(): ...

@pytest.fixture
def seeded_catalog(): ...

# Bad – one mega-fixture for everything
@pytest.fixture
def test_data():
    return {"product": ..., "catalog": ..., "user": ..., "config": ... }
```

Small fixtures compose. Big fixtures fight you.



Lab 7 — Unit Tests for the Catalog Core

80 min · open `labs/lab-07-unit-tests.md`

You'll build:

- `tests/conftest.py` — `sample_product`, `seeded_catalog` fixtures
- `tests/test_models.py` — Pydantic validation tests
- `tests/test_catalog.py` — `add/get/delete/queries/update`

End state: `pytest -q` green, ~17 tests passing.

Module 2

Parametrization, Mocking & Stubs

~40 min · then 80 min lab

@pytest.mark.parametrize — one test, many cases

```
@pytest.mark.parametrize("field,value,err_substring", [
    ("name", "", "at least 1 character"),
    ("price", -1, "greater than or equal to 0"),
    ("id", 0, "greater than or equal to 1"),
])
def test_rejects_invalid(field, value, err_substring):
    base = dict(id=1, name="X", category="c", price=10.0)
    base[field] = value
    with pytest.raises(ValidationError) as exc:
        Product(**base)
    assert err_substring in str(exc.value)
```

One test method → three reported tests. Add a row, add a test. No copy-paste.

Mock, stub, fake — pick the right one

	what it does	when to use
Stub	Returns a canned value	"give me back this list"
Mock	Records calls + returns canned values	"did I call POST with the right body?"
Fake	Lightweight working implementation	"in-memory dict instead of Postgres"
Spy	Wraps the real thing, records calls	"did the real method run?"

Today: **mocks** (verify behaviour) + a **fake** session (`MagicMock(spec=Session)`).

Mock HTTP — the right seam

Don't patch `requests.get` globally. Inject the session:

```
# In APIClient — already there from Day 2
def __init__(self, base_url, *, session=None):
    self._session = session or requests.Session()

# In the test
session = MagicMock(spec=requests.Session)
client = APIClient(base_url="http://test.local", session=session)
session.request.return_value = _mock_response(200, [ ... ])
```

Constructor injection > monkey-patching. Always.

MagicMock(spec= ...) — typo protection

```
session = MagicMock()                # bad – typos silently pass
session.request(method="GET", url=" ... ") # MagicMock auto-creates it

session = MagicMock(spec=requests.Session) # good
session.request( ... )                     # AttributeError: no such method
```

`spec=Session` makes the mock **reject calls that aren't on the real class**. One line of safety; never skip it.

Build a tiny mock-response helper

```
def _mock_response(status, payload):  
    resp = MagicMock(spec=requests.Response)  
    resp.ok = 200 ≤ status < 300  
    resp.status_code = status  
    resp.json.return_value = payload  
    resp.text = str(payload)  
    return resp  
  
session.request.return_value = _mock_response(404, {"detail": "missing"})
```

Three lines of helper that get used everywhere in the test file.

Test the retry logic with `side_effect`

```
session.request.side_effect = [  
    requests.ConnectionError("blip"),  
    requests.ConnectionError("blip"),  
    _mock_response(200, []),  
]  
  
assert client.list_products() == []  
assert session.request.call_count == 3
```

`side_effect = [a, b, c]` returns `a`, then `b`, then `c`. Raised exceptions in the list get raised. Perfect for testing retry semantics.

Integration tests — the other half

```
@pytest.fixture(scope="session")
def live_server():
    port = _free_port()
    proc = subprocess.Popen(
        [sys.executable, "-m", "uvicorn", "catalog.server:app", "--port", str(port)],
        stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
    _wait_for(f"http://127.0.0.1:{port}/health")
    yield f"http://127.0.0.1:{port}"
    proc.terminate(); proc.wait(timeout=5)

@pytest.mark.integration
def test_full_crud(live_server):
    client = APIClient(base_url=live_server)
    ...
```

`scope="session"` = one server for the whole test run. Don't re-boot per test.

Mark slow tests so they're opt-in

```
# pyproject.toml
[tool.pytest.ini_options]
markers = ["integration: tests that hit a live FastAPI server (slow)"]
```

```
pytest -m "not integration" -q    # fast feedback loop (~0.5s)
pytest -m integration             # full confidence (slower)
pytest                           # both (CI)
```

`--strict-markers` makes a typo'd marker fail loudly instead of running nothing.

Lab 8 — Test the `APIClient` (mocked + integration)

80 min · open `labs/lab-08-test-apiclient.md`

You'll build:

- Mocked unit tests for success + error mapping (parametrized)
- A retry-behaviour test using `side_effect`
- An `@pytest.mark.integration` class hitting a real server
- The `live_server` session-scoped fixture

The same mocking pattern is what we'll use on Day 4 to mock the LLM.

Module 3

Reporting & CI/CD

~40 min · then 80 min lab

Coverage — what is even tested?

```
pytest --cov --cov-report=term-missing
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
catalog/client.py	57	7	4	0	85%	68-69, 104-108
catalog/models.py	76	0	12	0	100%	
catalog/decorators.py	36	13	4	2	62%	24-35

- **Coverage measures lines executed, not behaviour verified.** 100% with no assertions = nothing.
- Look at the **Missing** column — that's your TODO list.

HTML report — for humans (and email)

```
pytest --html=report.html --self-contained-html
```

Open `report.html` in a browser:

- Sortable list of tests
- Each test's output captured
- Filterable by pass/fail/skip
- `--self-contained-html` inlines CSS so the file works when forwarded

Pair with `pytest-cov` and you've got the **test-suite dashboard**.

CI/CD for tests — the loop

```
push commit — trigger —> GitHub Actions
                                |
                                ├── install deps
                                ├── run pytest (+coverage +HTML)
                                ├── upload report artifact
                                └──  /  back to PR page
```

Goal: every PR shows green or red **before** human review.

`.github/workflows/tests.yml` — the bones

```
name: tests
on: [push, pull_request, workflow_dispatch]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
    matrix:
      python-version: ["3.10", "3.11", "3.12"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "${{ matrix.python-version }}", cache: pip }
      - run: pip install -e ".[dev]"
      - run: pytest --cov --html=report.html --self-contained-html
```

Upload the HTML report

```
- if: always()  
  uses: actions/upload-artifact@v4  
  with:  
    name: pytest-report-${{ matrix.python-version }}  
    path: report.html
```

`if: always()` uploads on failure too — that's the time you most want it.

Click any run on the GitHub Actions page → artifacts panel → download `report.html`.

Coverage in the GitHub UI

```
- if: always()  
  run: |  
    echo "### Coverage" >> $GITHUB_STEP_SUMMARY  
    python -c "import xml.etree.ElementTree as ET; \  
      r = ET.parse('coverage.xml').getroot(); \  
      print(f\"Line: {float(r.get('line-rate'))*100:.1f}%\")" \  
    >> $GITHUB_STEP_SUMMARY
```

`$GITHUB_STEP_SUMMARY` renders markdown on the run's summary page. Coverage at a glance, no clicking into artifacts.



Lab 9 — Reports + GitHub Actions

80 min · open `labs/lab-09-reports-ci.md`

You'll add:

- `--cov` + `--html` config in `pyproject.toml`
- `.github/workflows/tests.yml` — matrix across 3.10/3.11/3.12
- Push → green checkmark on GitHub

End of Day 3 → your repo matches `checkpoints/day-4-start/`.

End of Day 3

Tomorrow: add an LLM-powered agent — and use *exactly these same testing patterns* to validate the agent's tools and outputs.

Before you leave: push, see the green check, save the badge URL.